1. tokenize, filtration, stop word removal, stemming

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')

text = "The students are playing games happily!"

words = nltk.word_tokenize(text)
words = [w.lower() for w in words if w.isalpha()]
words = [w for w in words if w not in stopwords.words('english')]

ps = PorterStemmer()
result = [ps.stem(w) for w in words]

print("Tokens:", words)
print("Stemmed:", result)
```

2. ing, ed, s

```
words = ["playing", "walked", "books", "running"]

for w in words:
    if w.endswith("ing"):
        print(w, "Verb")
    elif w.endswith("ed"):
        print(w, "Past Tense Verb")
    elif w.endswith("s"):
```

```
        print(w, "Plural")
    else:
        print(w, "Unknown")
```

3. noun, verbs adjectives

```
sentence = "The smart boy plays cricket"

nouns = ["boy", "cricket"]
verbs = ["plays", "runs"]
adjectives = ["smart", "good"]

for w in sentence.split():
    if w in nouns:
        print(w, "Noun")
    elif w in verbs:
        print(w, "Verb")
    elif w in adjectives:
        print(w, "Adjective")
```

4. porterstemmer, snowball

```
import spacy
import pandas as pd
from nltk.stem import PorterStemmer, SnowballStemmer

words = ["playing", "studies", "running"]

p = PorterStemmer()
s = SnowballStemmer("english")
nlp = spacy.load("en_core_web_sm")

data = {
```

```
    "Word": words,
    "Porter": [p.stem(w) for w in words],
    "Snowball": [s.stem(w) for w in words],
    "Lemma": [nlp(w)[0].lemma_ for w in words]
}

print(pd.DataFrame(data))
```

5. naïve bayes

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

x = ["I love it", "Very good", "I hate it", "Very bad"]
y = ["Positive", "Positive", "Negative", "Negative"]

v = CountVectorizer()
X = v.fit_transform(x)

model = MultinomialNB()
model.fit(X, y)

pred = model.predict(X)

print("Predictions:", pred)
print("Accuracy:", accuracy_score(y, pred))
```

6. tf-idf score

```
from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["Python is easy", "Python is powerful", "NLP is interesting"]
```

```
v = TfidfVectorizer()
x = v.fit_transform(docs)

print("Words:", v.get_feature_names_out())
print("TF-IDF Scores:\n", x.toarray())
```

7. tf-idf, cosine similarity

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["Python programming", "Machine learning",
        "Natural language processing"]

query = ["Python learning"]

v = TfidfVectorizer()
x = v.fit_transform(docs + query)

scores = cosine_similarity(x[-1], x[:-1])[0]

for i in scores.argsort()[::-1]:
    print(docs[i], ":", round(scores[i], 2))
```